



Dime cómo puedo usar floorplanvlm en archicheck

Sí, pero con una aclaración importante: FloorPlanVLM no es la mejor pieza para “extraer geometría” directamente en ArchiCheck si tu entrada principal es PDF 2D vectorial. Te sirve mejor como capa semántica auxiliar sobre imágenes recortadas de regiones ambiguas, o como fallback cuando el parsing vectorial no logra inferir puertas/recintos. ^[1] ^[2] ^[3]

Cómo encaja en ArchiCheck

Yo lo usaría así:

1. **Primero tu pipeline vectorial determinístico:** PyMuPDF/OpenCV para muros, arcos, puertas, ventanas, textos y recintos.
2. **Luego FloorPlanVLM solo en zonas dudosas:** por ejemplo, un crop de una puerta que tu parser no reconoció, una nube de símbolos raros, o un recinto donde falta el label.
3. **Finalmente reconciliación:** si FloorPlanVLM sugiere “door” pero no hay soporte geométrico, no lo aceptas como verdad; lo marcas como hipótesis para revisión humana. Esto reduce al mínimo el riesgo de que el modelo “invente” geometría y te contamine el motor normativo. ^[2] ^[3] ^[1]

Forma práctica de usarlo

La implementación más razonable es tratar FloorPlanVLM como un modelo VLM de vectorización a JSON:

- Entrada: imagen rasterizada del plano o crop.
- Prompt: “Vectorize this floor plan into structured JSON with all walls, doors, windows, and rooms.”
- Salida: JSON con walls, doors, windows, rooms.
- Postproceso: validación geométrica tuya, snap a primitivos PDF, consistencia topológica, detección de contradicciones.
El repos público asociado al entrenamiento muestra precisamente este patrón de salida estructurada JSON para muros/puertas/ventanas/rooms. ^[3] ^[2]

Cómo integrarlo técnicamente

Opción A: inferencia remota / Hugging Face

Si usas una variante publicada en Hugging Face, el flujo es:

- Renderizas una página del PDF a imagen alta resolución.
- Haces crop por regiones relevantes si quieres ahorrar tokens/costo.
- Llamas al modelo con el prompt de vectorización.
- Parseas JSON.
- Mapeas resultados a tus entidades internas.

Un ejemplo de repos relacionado muestra uso con `transformers`, `Qwen2_5_VLForConditionalGeneration` y `AutoProcessor`, y otro repo/HF card usa inferencia vía API para devolver estructuras tipo `walls`, `room_polygons`, etc. ^[4] ^[3]

Opción B: self-host

Si tienes GPU, puedes correr una versión local basada en Qwen2.5-VL o similar, que es la familia que aparece en el material público de entrenamiento/inferencia. Eso te da control, evita costos por request, y te deja auditar prompts, temperatura y schema. ^[2] ^[3]

Qué le pediría exactamente

Para ArchiCheck, no lo usaría con un prompt abierto tipo “describe el plano”. Le daría un schema rígido:

- `walls`: id, start, end, thickness, curvature.
- `openings`: tipo, centro, ancho, orientación.
- `rooms`: label, polygon, area_hypothesis.
- `confidence`: por entidad.
- `evidence`: breve justificación textual opcional.

La idea es que FloorPlanVLM no sea juez, sino **generador de candidatos**; tu motor geométrico sigue siendo el árbitro final. El propio repos de entrenamiento enfatiza salida estructurada en JSON para vectorización. ^[3] ^[2]

Lo que sí y lo que no

Sí lo usaría para:

- puertas que tu parser no detecta,
- habitaciones sin nombre,
- casos donde el achurado o la convención CAD confunde reglas determinísticas,
- fallback semántico cuando el PDF está rasterizado o degradado. ^[5] ^[1] ^[2]

No lo usaría para:

- reemplazar tu lectura vectorial nativa,

- decidir cumplimiento normativo,
- confiar en métricas geométricas sin validación topológica,
- inferir muros con precisión milimétrica si el PDF original ya tiene vectores claros. ^{[6] [1] [2]}

Recomendación concreta

Mi recomendación para ArchiCheck es esta:

- **Arquitectura:** tu parser vectorial sigue siendo la base.
- **FloorPlanVLM:** solo como módulo de rescate para símbolos/recintos ambiguos.
- **Validación:** toda salida del VLM se somete a tu lógica geométrica y a revisión humana.

Si quieres, te puedo dar el siguiente paso en formato muy operativo: **un diseño de integración en 3 funciones Python** (render_page, query_floorplanvlm, merge_with_geometry) con un prompt y un schema JSON listos para usar en ArchiCheck.

**

1. <https://arxiv.org/html/2602.06507v1>
2. <https://huggingface.co/manitocross/floorplan-vlm-training>
3. <https://huggingface.co/minemaster01/qwen25-vl-3b-floorplan-grpo>
4. <https://huggingface.co/rikkhoffbauer2/floorplan-parser>
5. <https://arxiv.org/abs/2409.12842>
6. <https://meshinglab.dcc.uchile.cl/publication/pizarro-2022104348/>
7. <https://github.com/miladmirzazadeh/FloorPlanVLM>
8. <https://huggingface.co/sabaridsnfuji/FloorPlanVisionAIAdaptor>
9. https://github.com/bu-air-lab/floor_plan_VLM
10. <https://ingcivil.uchile.cl/dam/jcr:7dad517d-f23e-484c-b9a6-9a8b6d981faf/Presentacion Pablo Pizarro R 24 julio 2024.pdf>
11. <https://www.vlm.run/solutions/construction>
12. <https://repositorio.uchile.cl/bitstream/handle/2250/196842/Wall-polygon-retrieval-from-architectural-floor-plan-images-using-vectorization-and-Deep-Learning-methods.pdf?sequence=1&isAllowed=y>
13. https://papers.cumincad.org/data/works/att/caadria2024_514.pdf
14. https://www.u-cursos.cl/ingenieria/2023/1/CC7910/1/material_docente/bajar?bajar=1&id=6423841
15. <https://patents.google.com/patent/US20220148327A1>